[/]

Polling is the Vulnerability /

A case for Event Driven Cloud Detection and Response

VB2026 - 16/10/2026 · Santiago Abastante

Slides



/intro

Santi Abastante

About Me
- Former Police Officer
- Cloud Security Engineer
- Incident Responder
- Digital Nomad



[00]

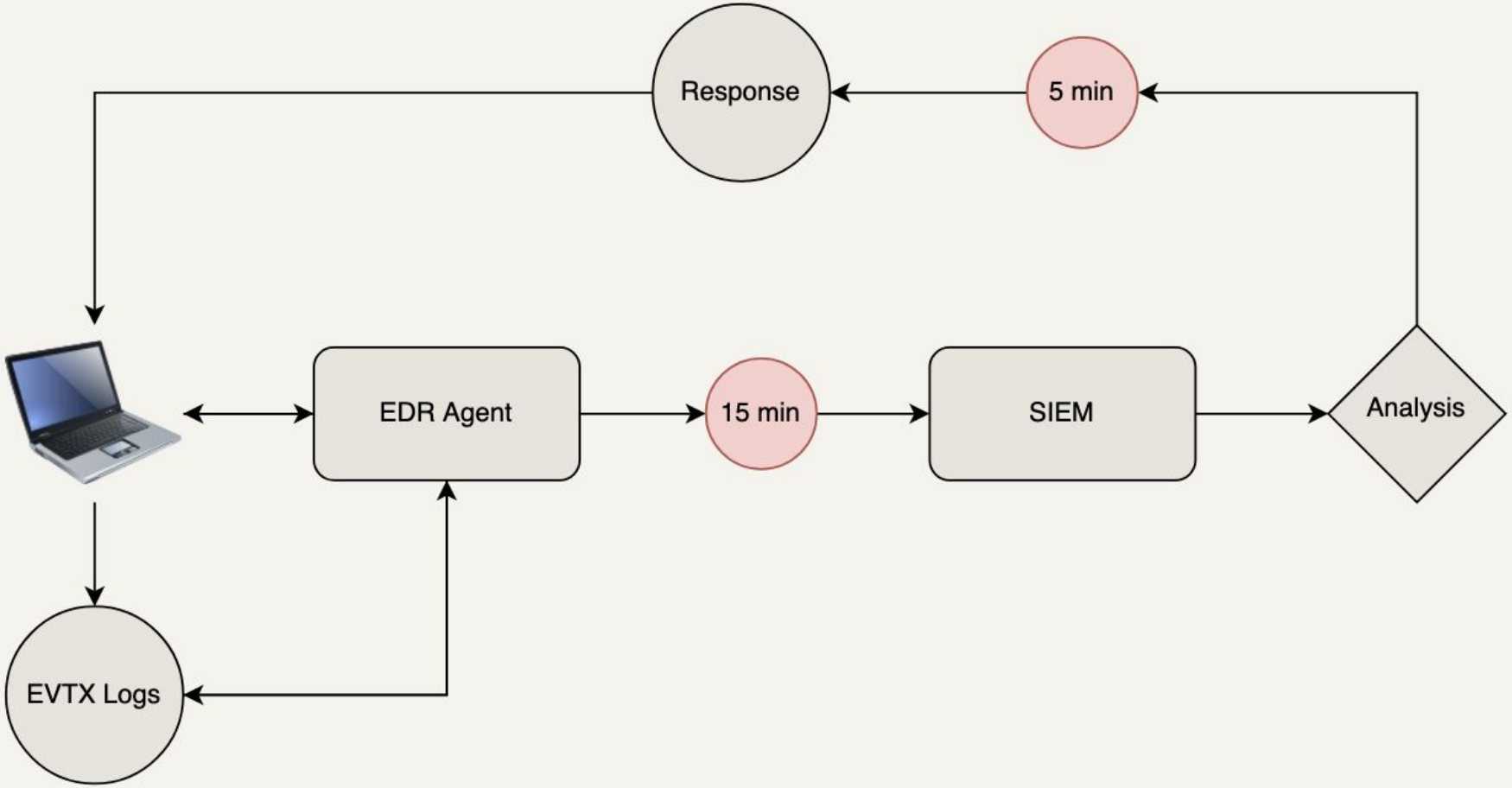
Disclaimer:

‘Open-source ideas, not products.’

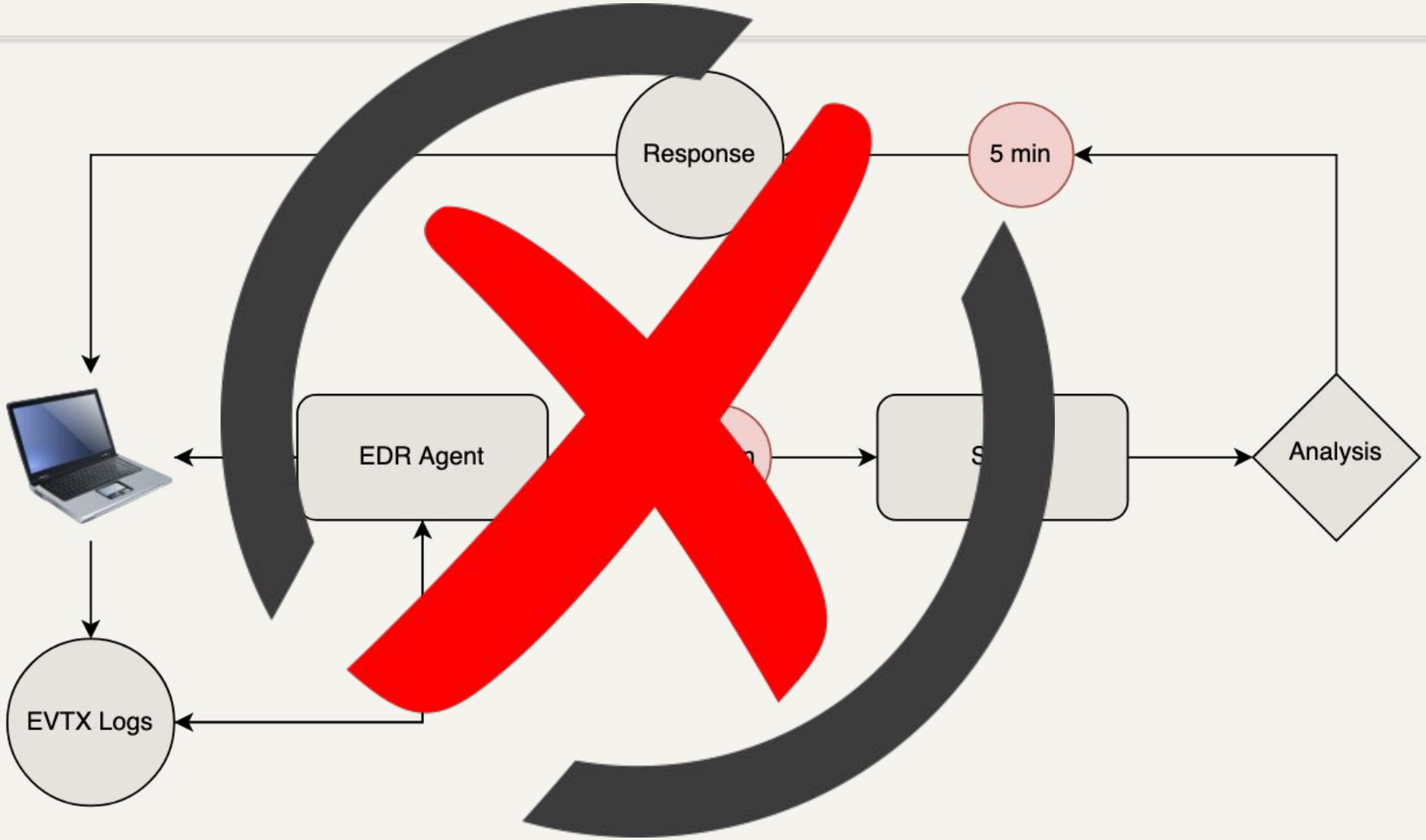
[01]

Intro

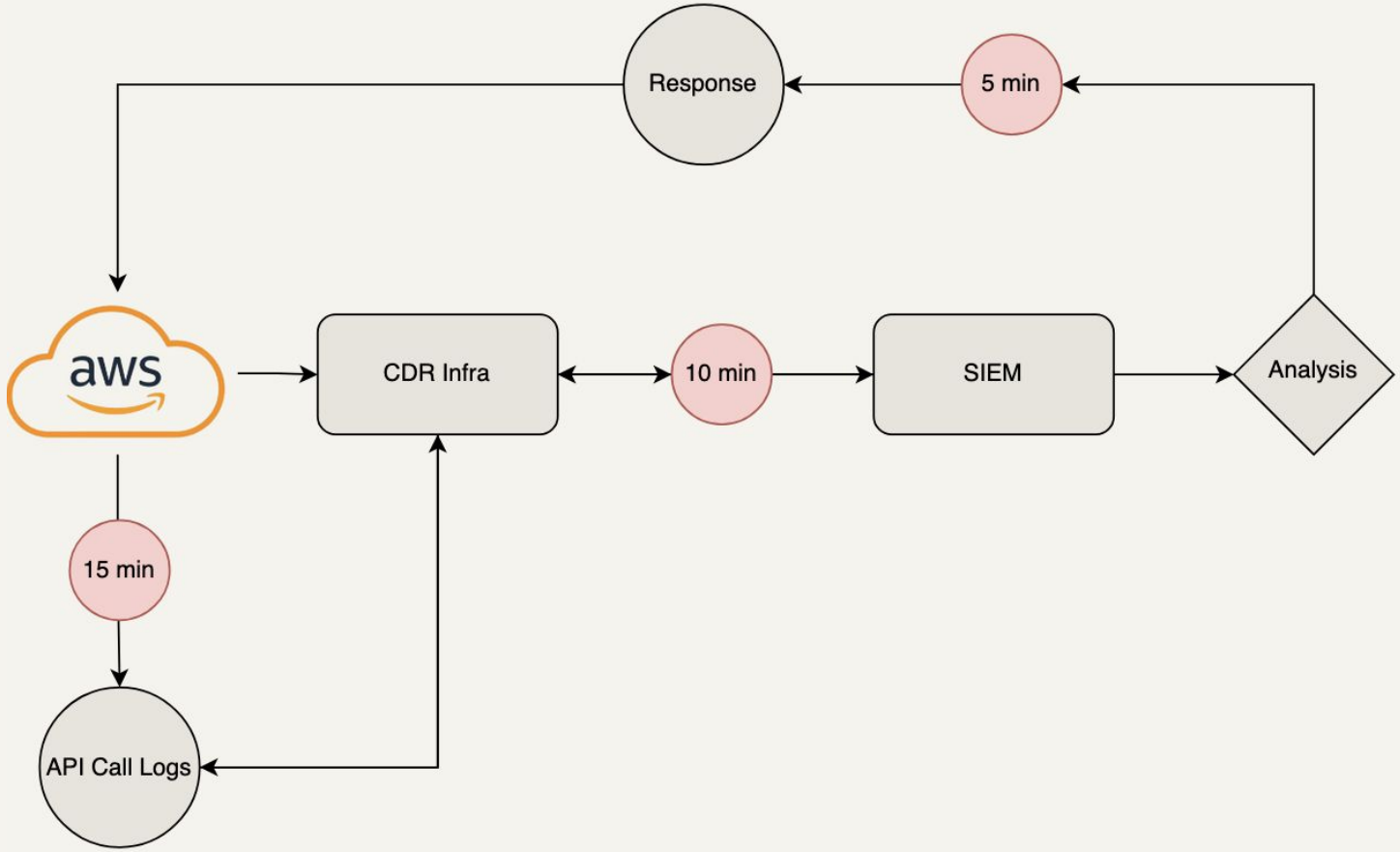
Imagine this EDR



How does an EDR Works? (4)

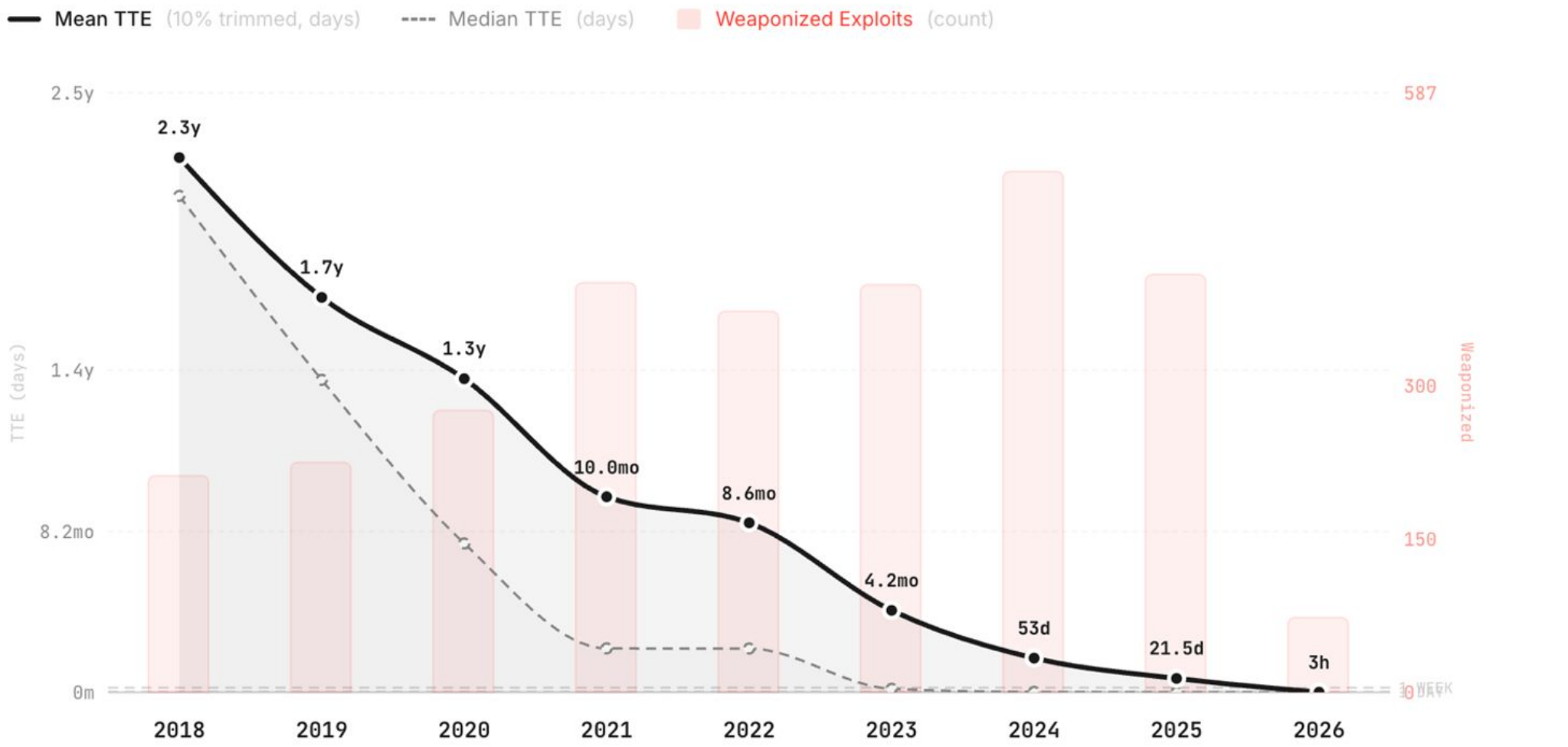


How traditional Cloud Detection and Response Works



From Vulnerability to Exploitation

TTE measures the gap between CVE public disclosure and first confirmed in-the-wild exploitation. Zero = same-day.



Based on 3,500+ confirmed-exploited CVEs (CISA KEV + VulnCheck KEV, with VulnCheck XDB timestamps for early-year CVEs)

zerodayclock.com

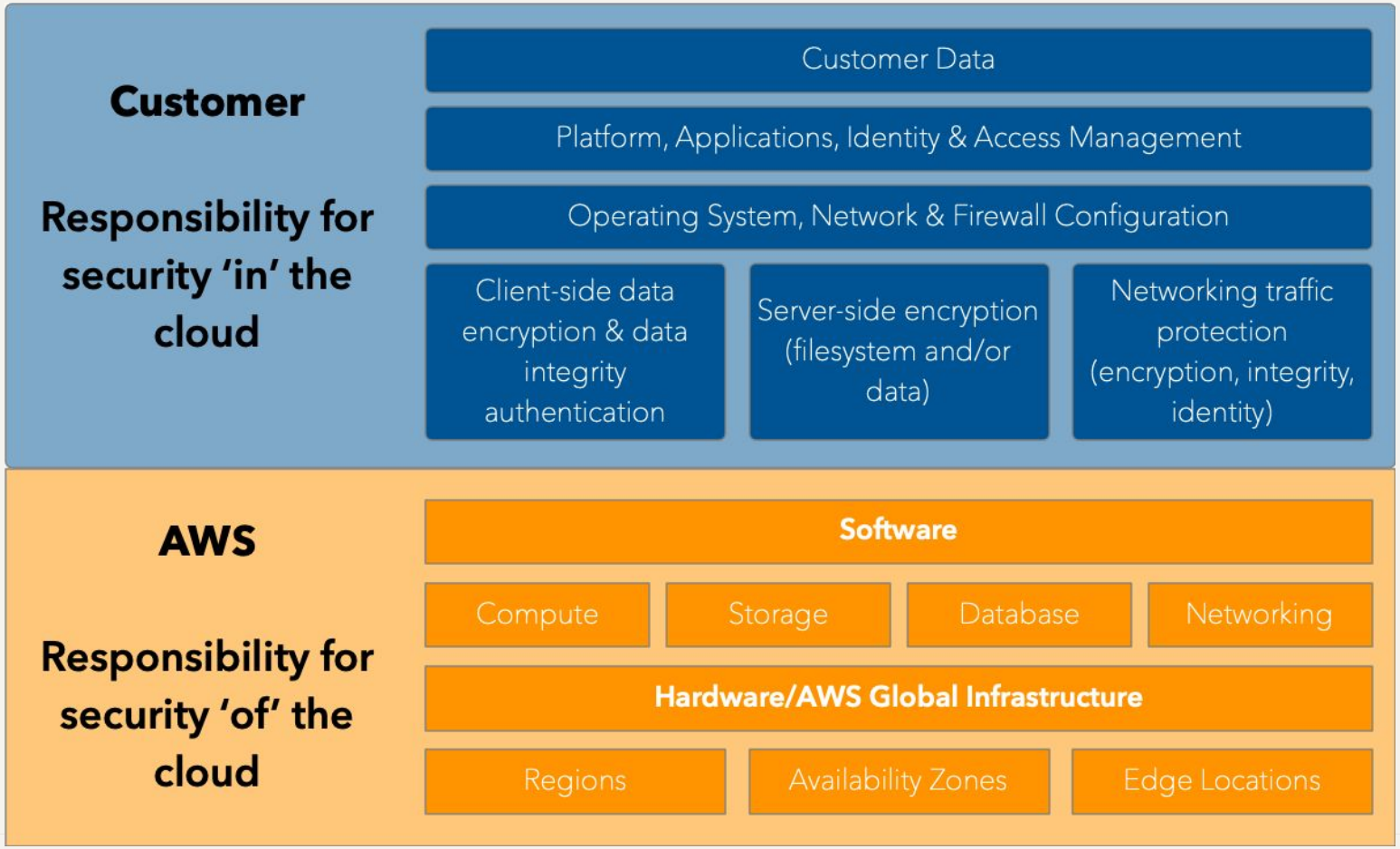
> **”All we have to decide is what to do with the time that is given us.”**
>
> *Gandalf - Lord of The Rings*

— dbnz/

[02]

AWS Logging and Monitoring

The Share Responsibility Model



> ”Every Service in AWS is an API, and every API Call executed is logged in Cloudtrail”

— dbnz/

Cloudtrail

- **Every Service in AWS is an API,**
- **Cloudtrail Logs every API Call,**
- Can be consumed in 3 ways, each one with different delays and tradeoffs,
- Can be consumed using S3, Cloudwatch or Event Bridge,
- Most important Forensic Artifact for IR,
- **Some API Call require additional configurations.**



Cloudtrail Log examples



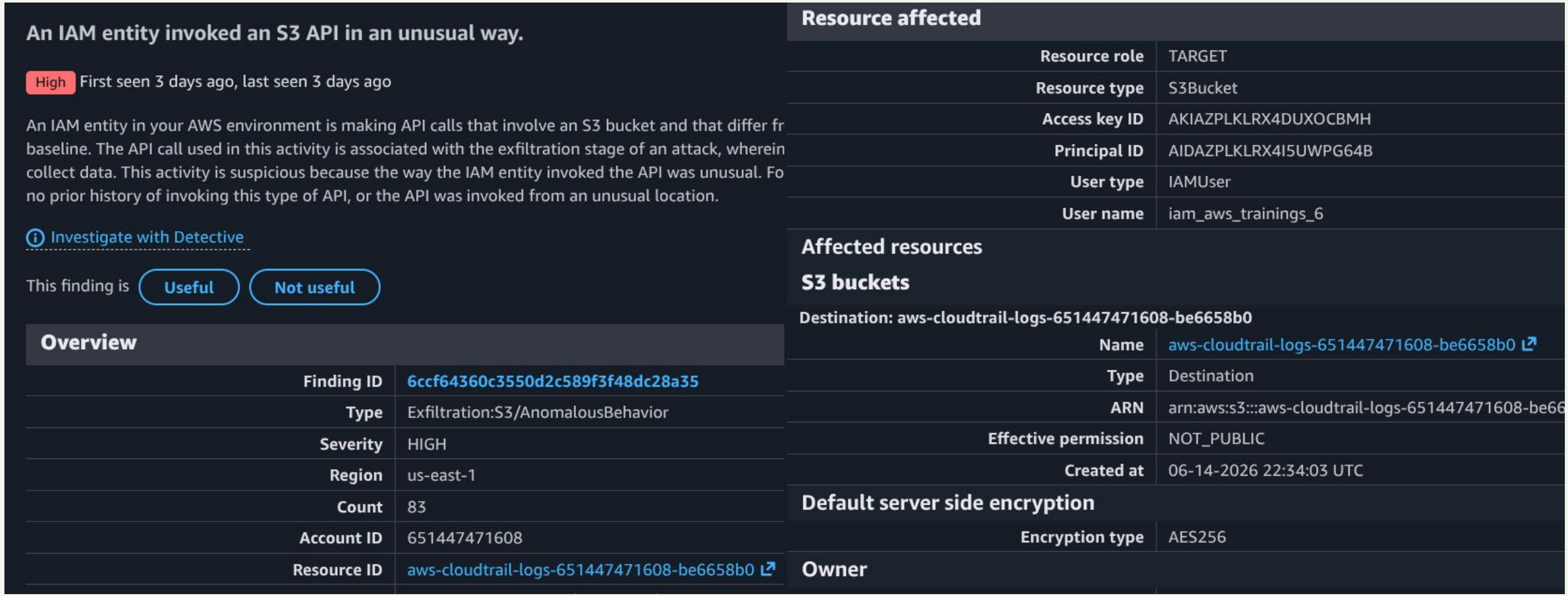


Guardduty

- **AWS Threat Detection and Response**,
- **Correlates events from API Logs, Network Logs, DNS, Logs, etc,**
- **Cannot create custom rules**,
- Uses IA for detection;
- **Delays: Up to 15 Min**,
- It's getting better every year.



Guardduty finding Example



An IAM entity invoked an S3 API in an unusual way.

High First seen 3 days ago, last seen 3 days ago

An IAM entity in your AWS environment is making API calls that involve an S3 bucket and that differ fr
baseline. The API call used in this activity is associated with the exfiltration stage of an attack, wherein
collect data. This activity is suspicious because the way the IAM entity invoked the API was unusual. Fo
no prior history of invoking this type of API, or the API was invoked from an unusual location.

Investigate with Detective

This finding is Useful Not useful

Overview

Finding ID	6ccf64360c3550d2c589f3f48dc28a35
Type	Exfiltration:S3/AnomalousBehavior
Severity	HIGH
Region	us-east-1
Count	83
Account ID	651447471608
Resource ID	aws-cloudtrail-logs-651447471608-be6658b0

Resource affected

Resource role	TARGET
Resource type	S3Bucket
Access key ID	AKIAZPLKLRX4DUXOCBMH
Principal ID	AIDAZPLKLRX4I5UWPG64B
User type	IAMUser
User name	iam_aws_trainings_6

Affected resources

S3 buckets

Destination: aws-cloudtrail-logs-651447471608-be6658b0

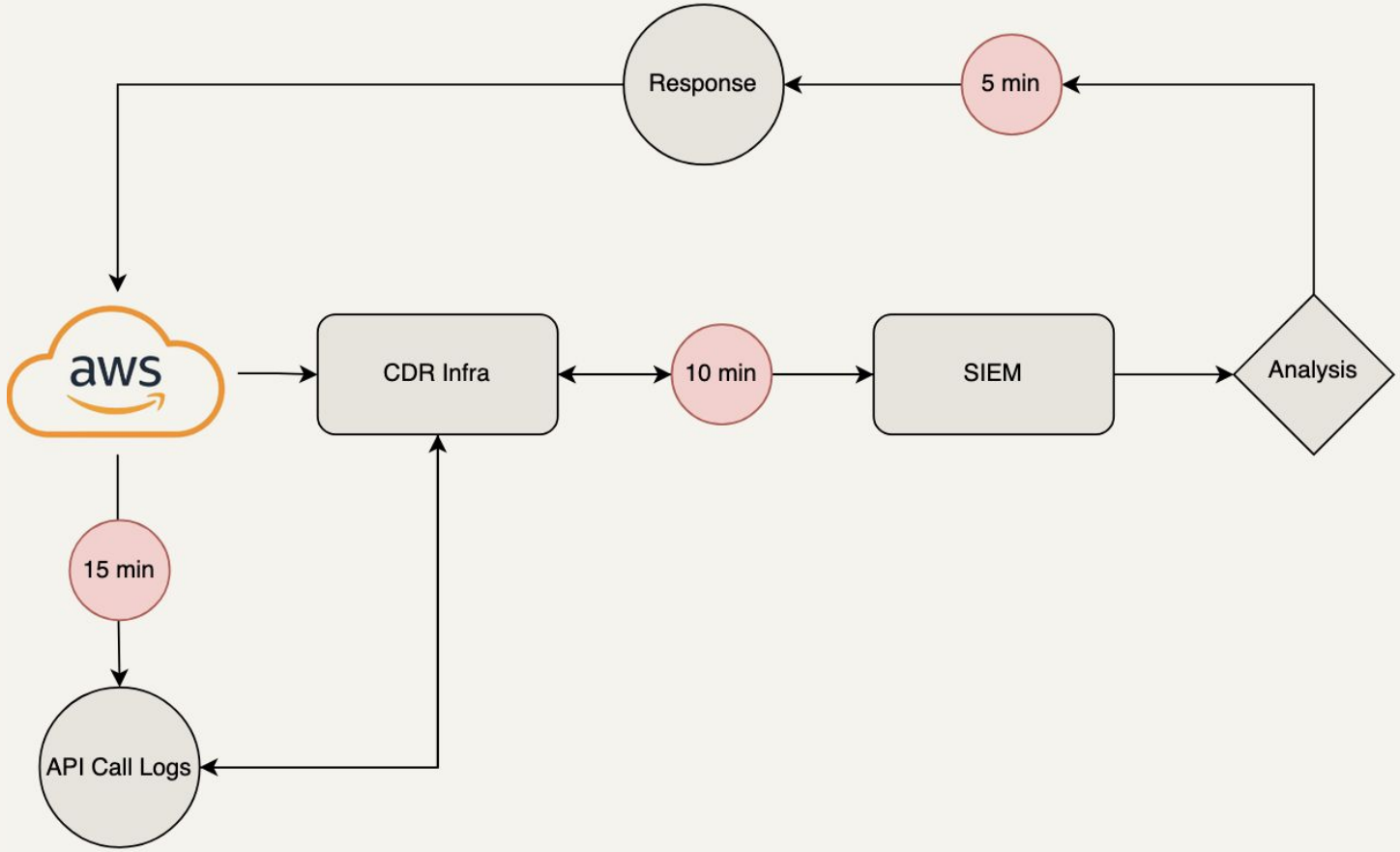
Name	aws-cloudtrail-logs-651447471608-be6658b0
Type	Destination
ARN	arn:aws:s3:::aws-cloudtrail-logs-651447471608-be66
Effective permission	NOT_PUBLIC
Created at	06-14-2026 22:34:03 UTC

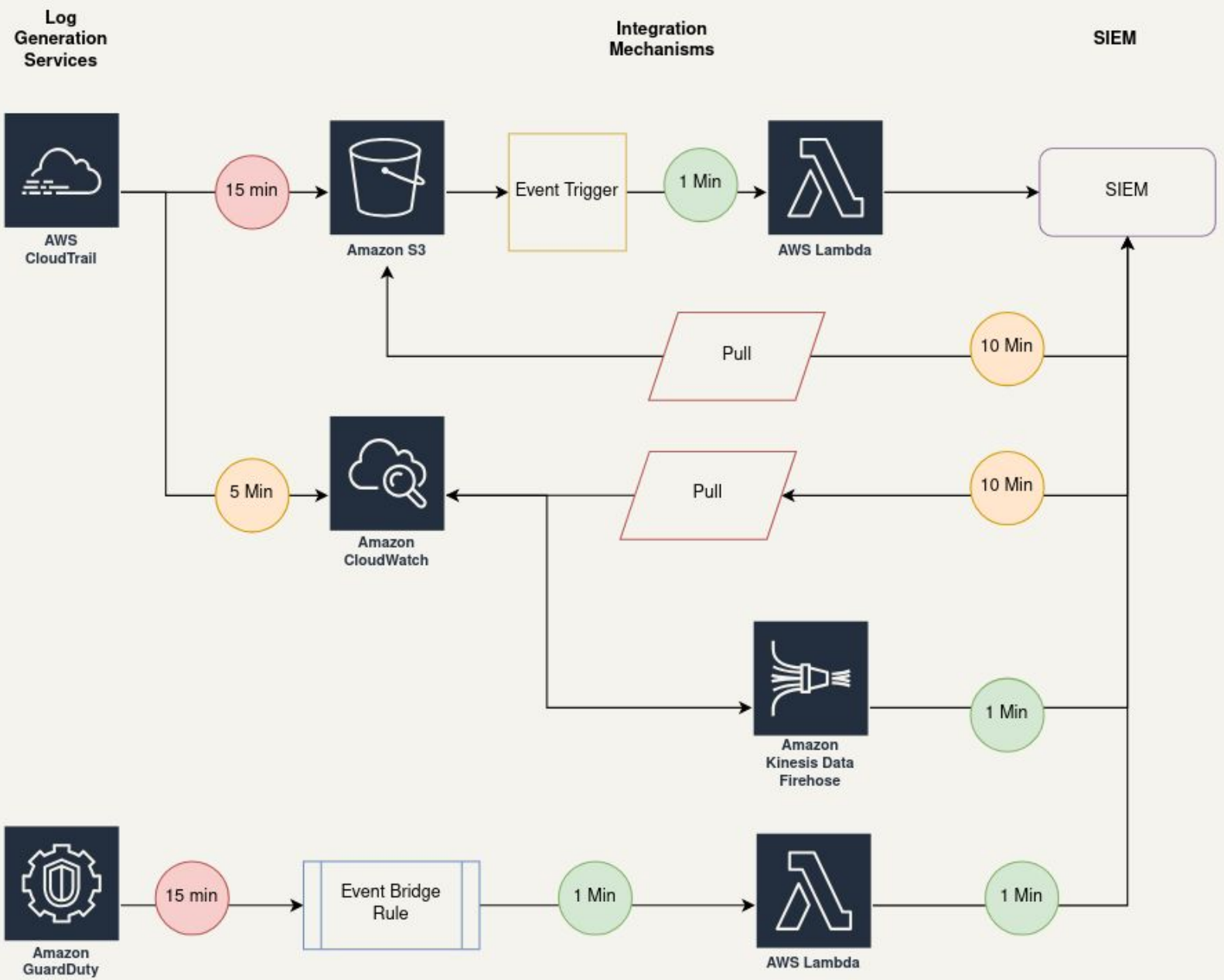
Default server side encryption

Encryption type	AES256

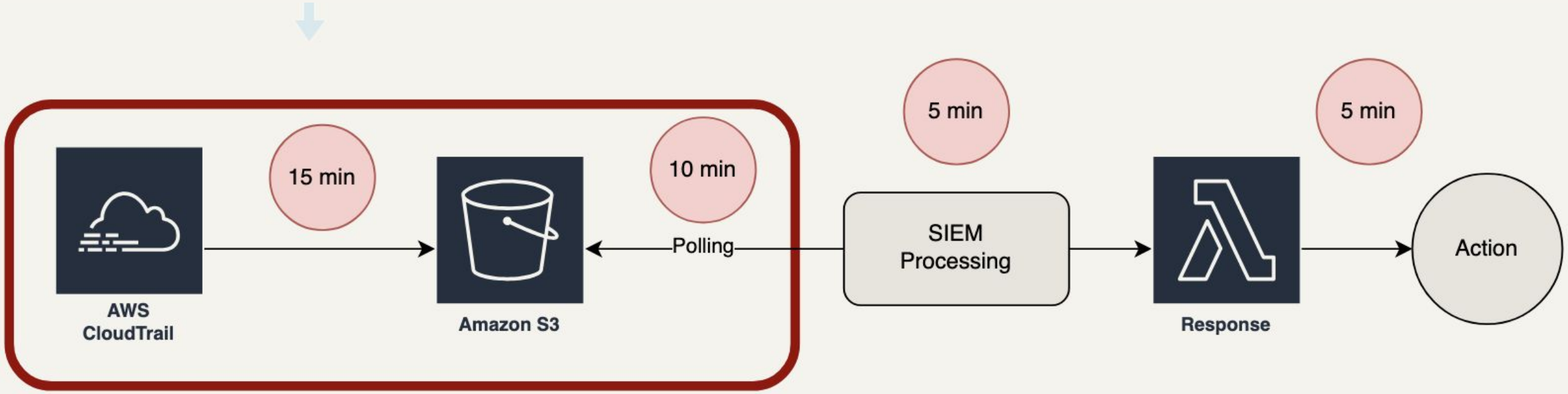
Owner

How traditional Cloud Detection and Response Works




Log Generation Services
Integration Mechanisms
SIEM
AWS CloudTrail
15 min
Amazon S3
Event Trigger
1 Min
AWS Lambda
SIEM
Pull
10 Min
5 Min
Amazon CloudWatch
Pull
10 Min
Amazon Kinesis Data Firehose
1 Min
Amazon GuardDuty
15 min
Event Bridge Rule
1 Min
AWS Lambda
1 Min

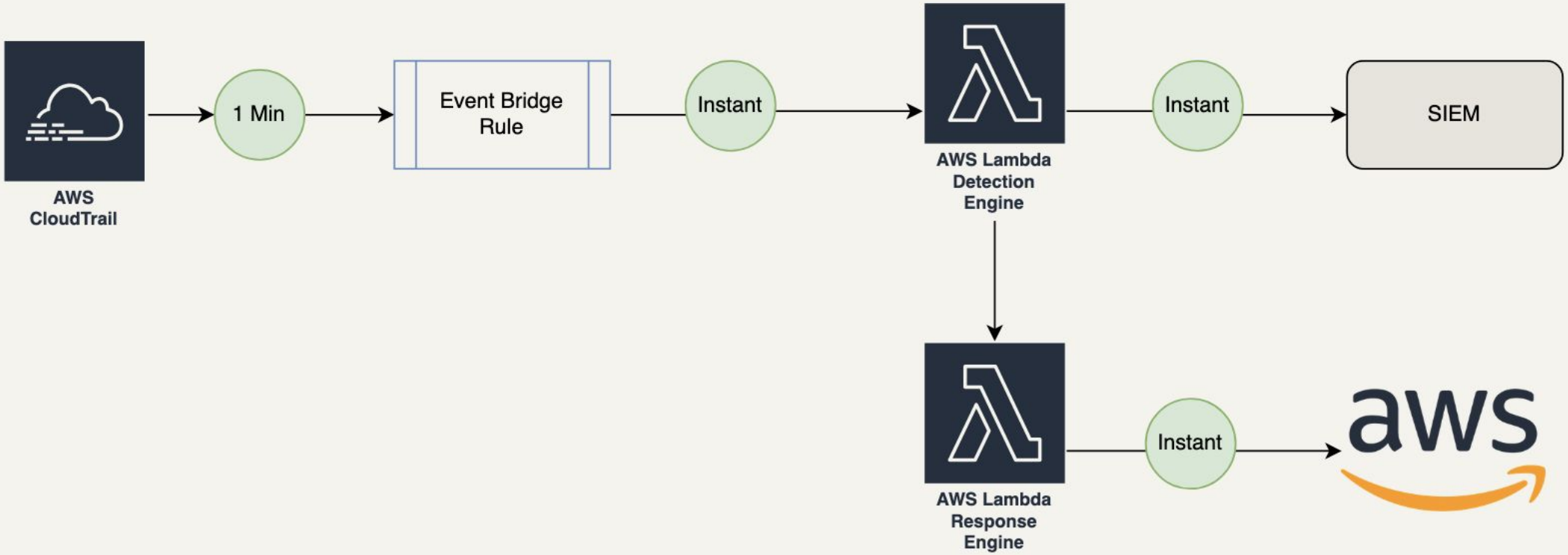
Traditional SIEM Detection and Response Architecture



Traditional SIEM Detection and Response Architecture



Proposed AWS Detection and Response Architecture



[03]

OpenCDR

Prior Work: Dredge and the Incident Response Perspective

README Contributing MPL-2.0 license Security

Dredge - 1.1.4

CI / Release passing License MPL 2.0 coverage ≥80%

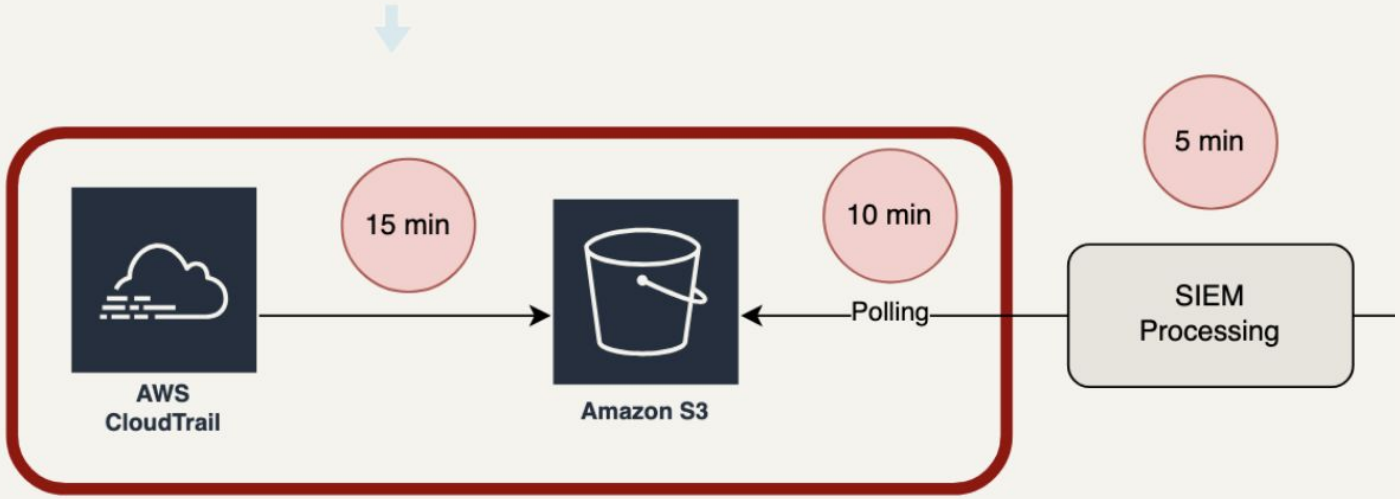
⚡ Cloud log collection, threat hunting, and rapid response... pa' la hinchada ⚡

Dredge is a cloud incident-response and threat-hunting toolkit — a Python library **and** a CLI — for **AWS, Kubernetes, GitHub, and GCP**. It's built for moving fast when you don't have all the plumbing ready at 3AM: collect logs, hunt through them, and contain — from one tool.

Git Repo

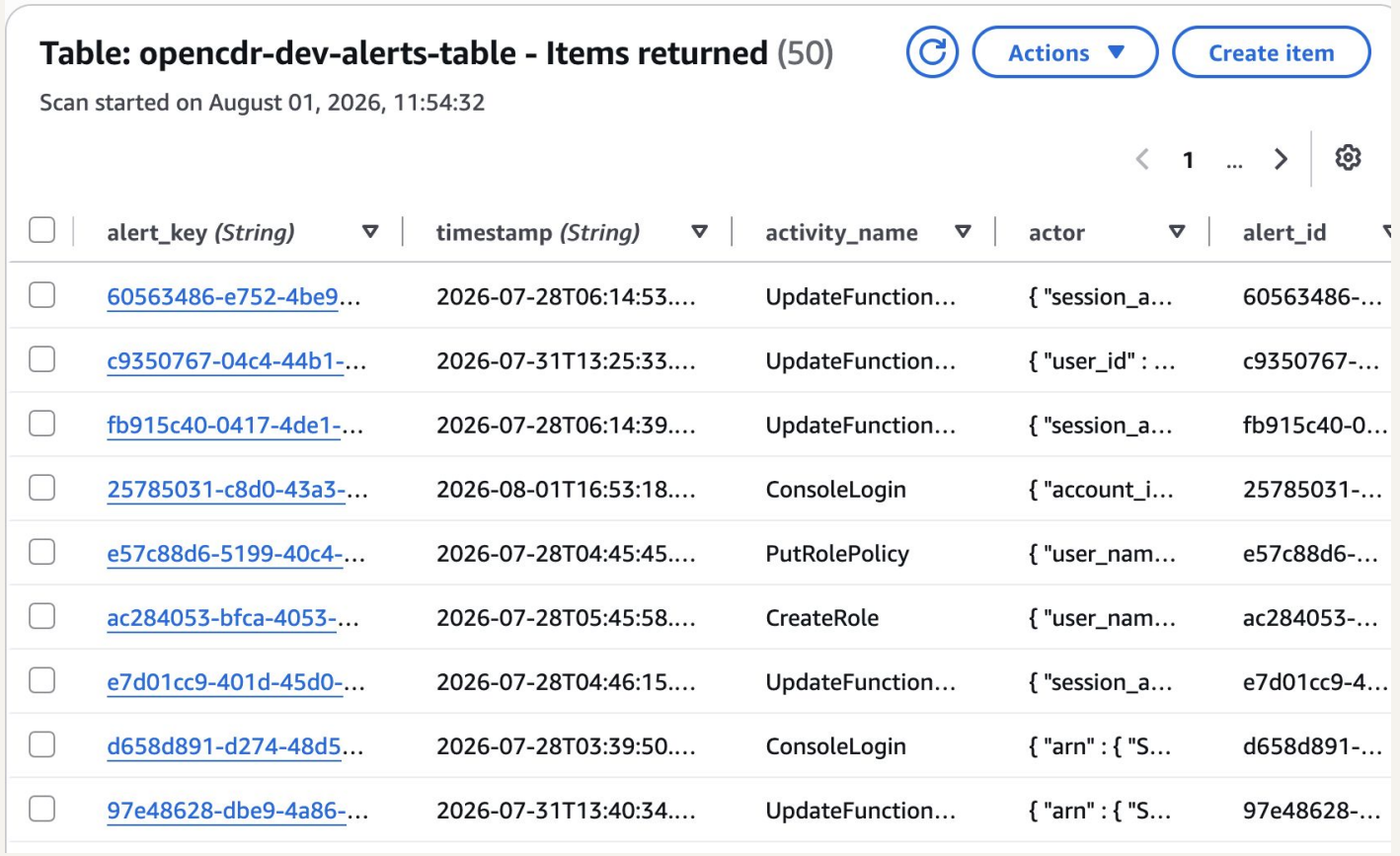
OpenCDR Principles: No polling

- **100% Event Driven**
- EventBridge delivers events to the processor Lambda.
- DynamoDB Streams deliver signal writes to the alerter Lambda.
- **There is no component that wakes up on a timer and asks "is there anything new?".**
- **This is the most important structural difference from conventional SIEM architectures.**
- **Near real time.**



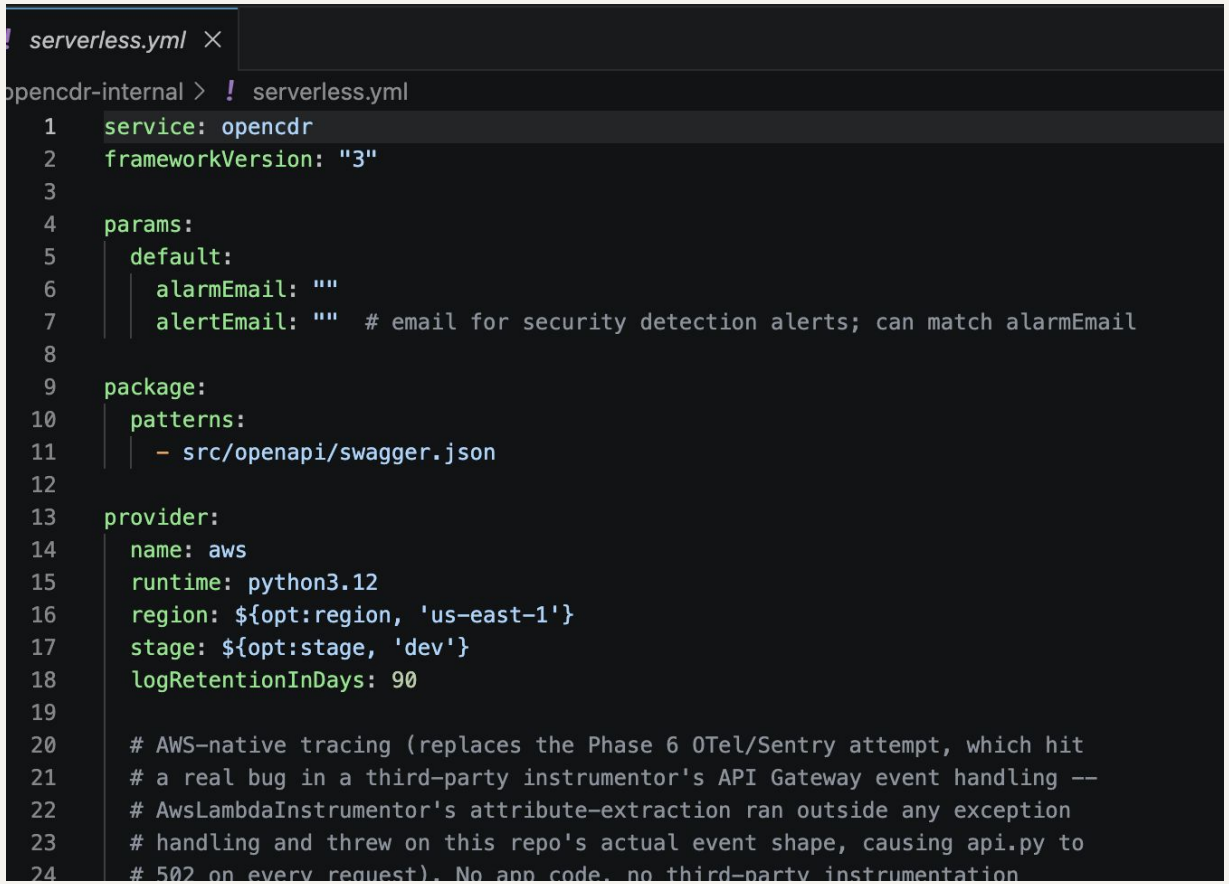
OpenCDR Principles: Explicit detection state.

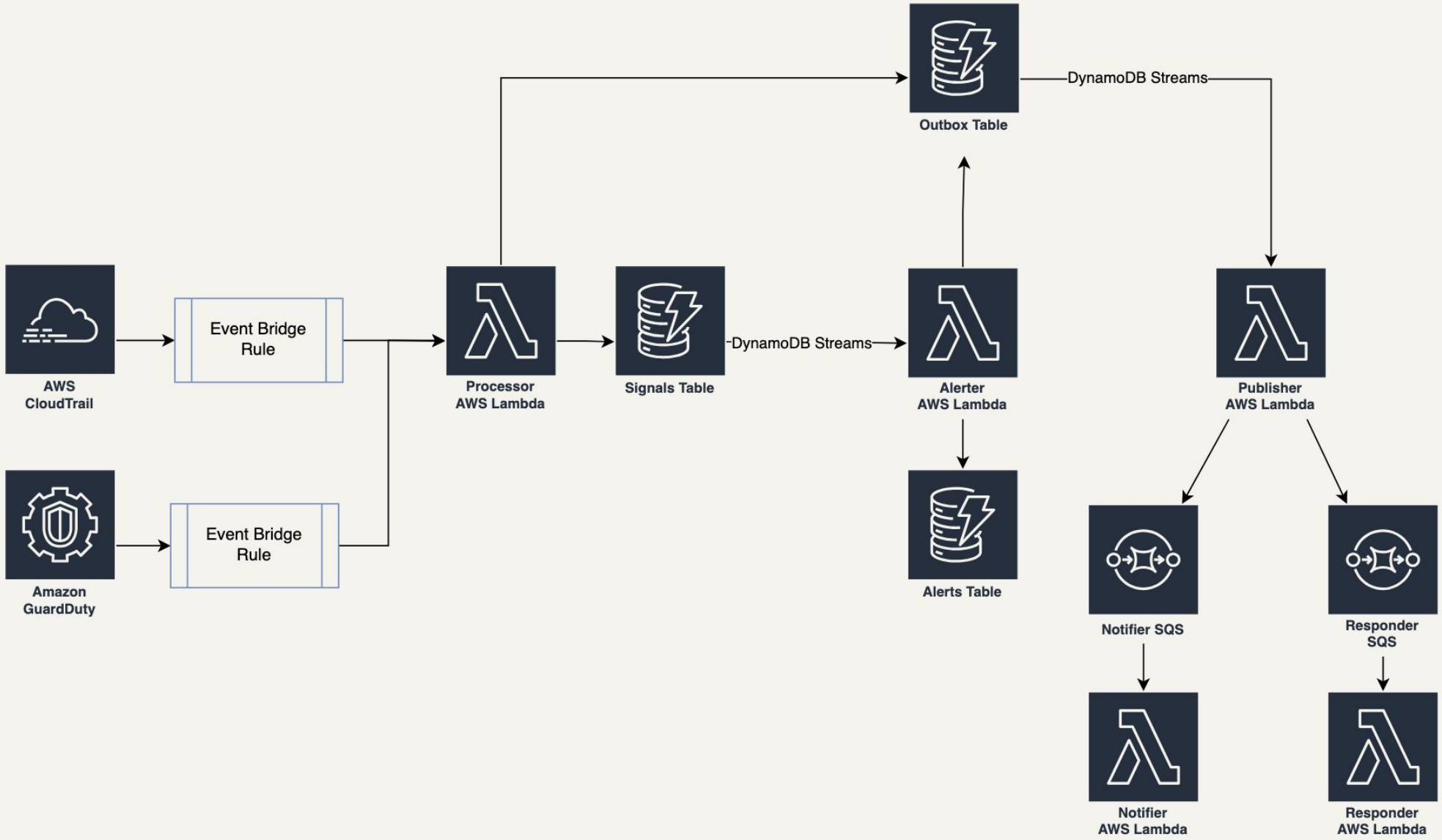
- Detection results—signals—are stored as first-class records in DynamoDB with a stable identity derived from the CloudTrail event ID.
- **Every detection is addressable, queryable, and auditable** independently of the detection rule that produced it.
- **Correlation operates over stored signals**, not over a stream of raw events that may or may not be available.
- **The state of the detection system at any point in time is fully inspectable.**



OpenCDR Principles: You OWN the infrastructure.

- **The engine is fully deployed in your AWS Organization**.
- Code is 100% open source.
- Signals and Correlation rules are 100% Open Source.
- You manage the Serverless IaC deployment.
- **All the telemetry stays within your AWS Tenant**
- Build In API layer provided.




Outbox Table
DynamoDB Streams
AWS
CloudTrail
Event Bridge
Rule
Processor
AWS Lambda
Signals Table
DynamoDB Streams
Alerter
AWS Lambda
Publisher
AWS Lambda
Amazon
GuardDuty
Event Bridge
Rule
Alerts Table
Notifier SQS
Responder
SQS
Notifier
AWS Lambda
Responder
AWS Lambda

Signal and Correlation Rules

- **Signal rules match a single normalized event against a list of conditions** (field/op/value — equals, contains, regex, etc.); when all conditions pass, processor writes one signal. This is the atomic detection unit — e.g. "IAM access key created."

- Correlation rules don't look at raw events at all — **they group signals by a field** (group_by, e.g. actor.user_name)

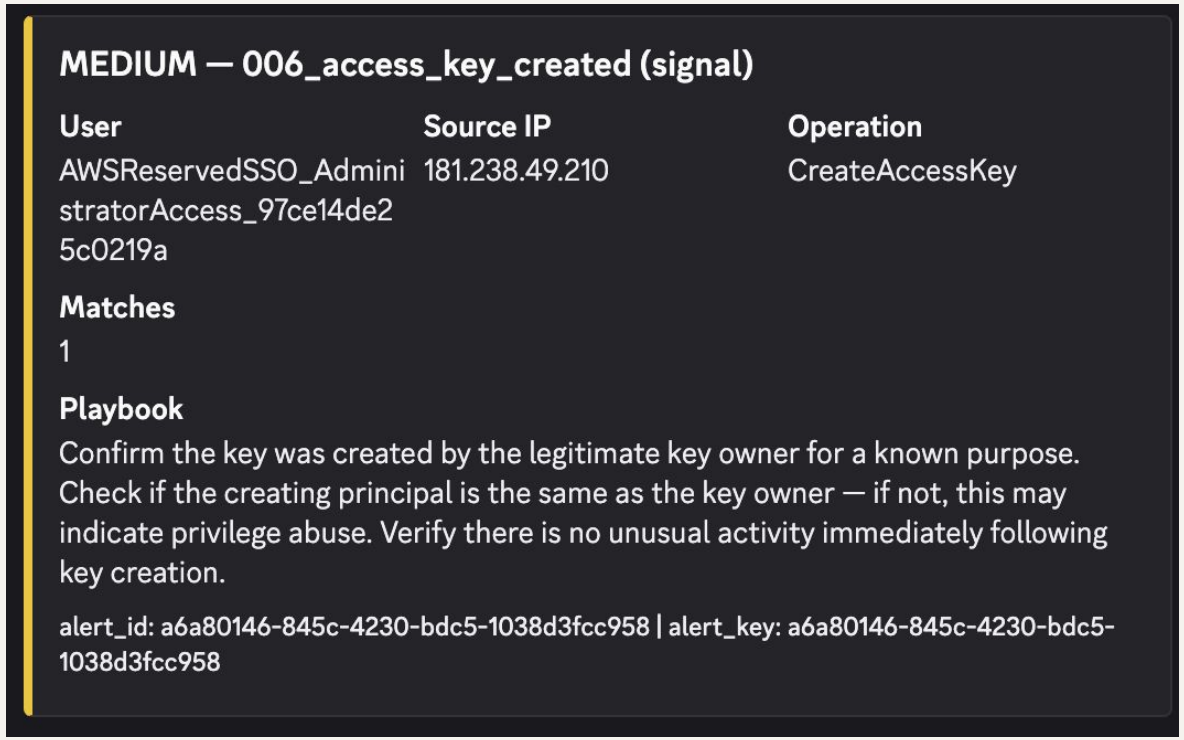
- **Rules are data, not code** — both kinds live as rows in a DynamoDB table.

- **Response_module is the bridge to automated response**


```
{
 "rule_kind": "signal",
 "rule_id": "006_access_key_created",
 "conditions": [
  {
   "field": "activity_name",
   "op": "equals",
   "value": "CreateAccessKey"
  },
  {
   "field": "api.service",
   "op": "equals",
   "value": "iam.amazonaws.com"
  }
 ],
 "created_by": "opencdr",
 "description": "New IAM access key created. Attackers create access keys for pe
 "enabled": true,
 "notify": true,
 "playbook": "Confirm the key was created by the legitimate key owner for a know
     the key owner — if not, this may indicate privilege abuse. Verify there is
 "response_module": "disable_access_key",
 "severity": "MEDIUM",
 "timestamp": "2026-04-03T16:13:28.302445+00:00",
```

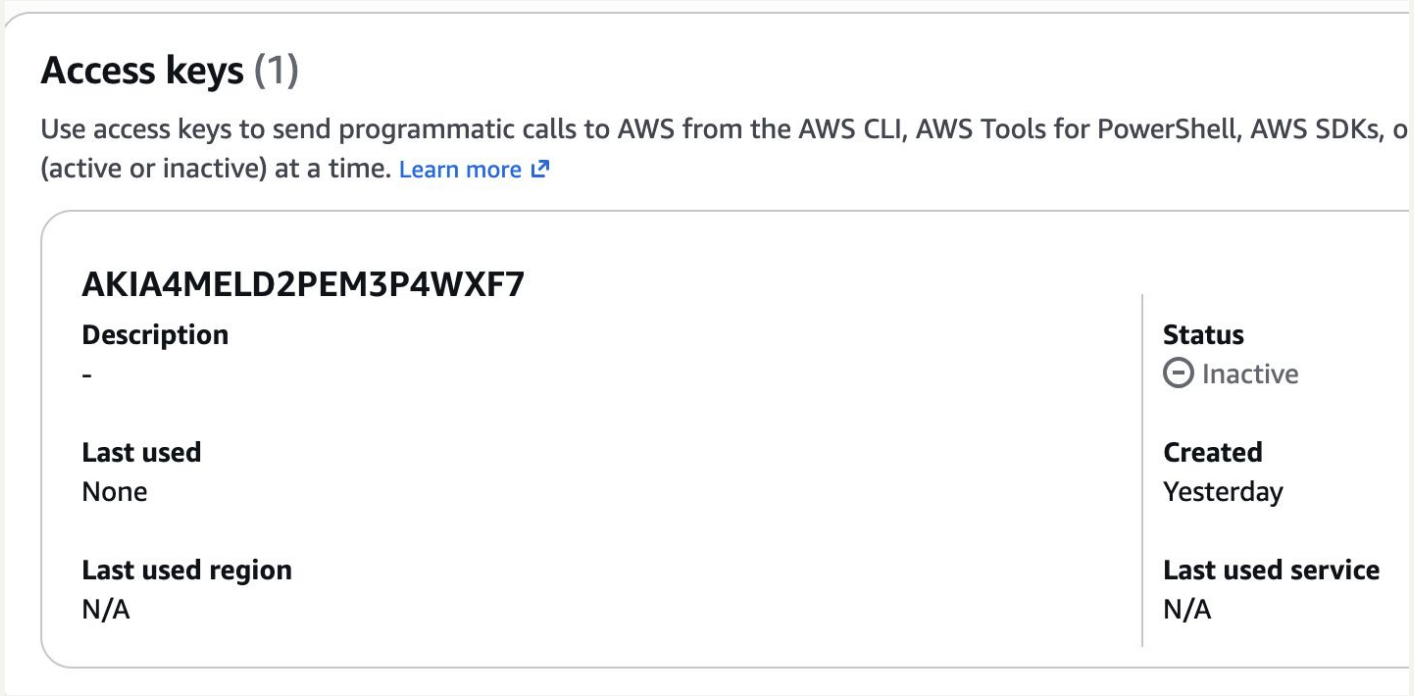
Notifications

- **Multi-channel by design** — one alert item can fan out to Slack, Discord, Email (via SNS), Security Hub (ASFF findings), Jira, and generic webhooks.
- All from the **same notifier Lambda** reading off the outbox-fed NotificationsQueue.
- **Severity drives both color and routing** — each channel builder maps severity to a visual signal (Slack attachment color, Discord embed color int),



Automated Incident Response

- **Dispatch table, not a giant if/else** — a rule's response_module string (e.g. disable_access_key) is looked up in a handler dict; each handler knows how to pull the fields it needs.
- **The "how" is delegated to a separate library** the actual AWS mutation (revoke keys, disable a user, etc.) is executed by dredge.
- **Cross-account by design.**
- A circuit breaker sits in front of every action
- **Every outcome is logged as structured, auditable fact.**



[/]

Demo/

Real Time Threat Detection and Response with OpenCDR

OpenCDR



One does not simply patches everything

- Assume **Zero-Day Discovery** Is Commodity.
- Reduce Blast Radius.
- **Detect Exploitation**, Not Just Vulnerabilities.
- **Automate Response**.
- Raise Human Awareness!



[/]

Questions?/

A Case for Event-Driven Cloud Detection and Response

23/09/2026 · Santiago Abastante

Linkedin: Santi Abastante - sabastante

Contact

OpenCDR

Dredge